

PRACTICAL 9

In-Memory Caching and Query Optimization

Student Name: Prisha Kalola

Student ID: 24DIT026

Semester: 5th Semester

CO/PO: CO2, CO4 / PO3, PO5

Course: Advanced Web Development Framework (ITUE301)

1. Aim

To implement server-side in-memory caching using node-cache and measure its impact on API response time and query performance.

2. Objectives

- Add in-memory caching to the Task Management Express backend using node-cache.
- Cache responses for GET /tasks and GET /tasks/:id with a 60-second TTL.
- Implement automatic cache invalidation on POST, PUT, and DELETE operations to prevent stale data.
- Expose cache hit/miss metrics via a debug endpoint (/debug/cache-stats).
- Empirically measure and compare API response times between cached (HIT) and uncached (MISS) requests.

3. Connection with Previous Practicals

Practical 9 optimizes the backend REST API built in Practicals 4-8 by introducing a high-performance in-memory caching layer before database calls.

4. Prerequisites

- Practicals 4-8 completed working Node/Express/MongoDB backend with JWT authentication.
- Installed package: node-cache.
- Postman or Thunder Client for measuring response latency.

5. Theory Concepts

5.1 In-Memory Caching Architecture

Repeated database reads at scale are computationally expensive. In-memory caching stores frequently accessed data in application RAM. On GET /tasks, Express checks node-cache first. On a CACHE HIT, data is returned instantly (~2ms) without touching MongoDB. On a CACHE MISS, MongoDB is queried, stored in cache, and returned.

5.2 Cache Invalidation Strategy

Cache invalidation is essential to maintain data correctness. Whenever a write operation (POST, PUT, DELETE) modifies MongoDB, `cache.del('all_tasks')` is executed so subsequent reads fetch fresh data.

6. Problem Definition

Implement node-cache in Express backend. Cache GET /tasks (all tasks) and GET /tasks/:id (single task) with 60s TTL. Invalidate cache on write operations, expose hit/miss statistics, and document response time improvements.

7. Tools / Technology

- Node.js v18+ & Express.js
- node-cache package
- MongoDB & Mongoose
- Postman / Thunder Client

8. Lab Session — Step-by-Step Implementation

Step 1: Install node-cache

```
npm install node-cache
```

Step 2: Shared Cache Module (cache.js)

Created `cache.js` with NodeCache instance (stdTTL: 60) and hit/miss counter helpers.

Step 3: Route Integration & Invalidation (server.js)

Updated GET /tasks to check `cache.get('all_tasks')` and set X-Cache header. Added `cache.del('all_tasks')` in POST, PUT, and DELETE handlers.

9. Screenshot Evidence — Practical 9 Implementation

VS Code - cache.js (node-cache shared instance)

```
const NodeCache = require('node-cache');  
const cache = new NodeCache({ stdTTL: 60, checkperiod: 120 });  
  
let hitCount = 0; let missCount = 0;
```

Figure 1: VS Code cache.js module defining node-cache instance and stats counter

Postman - GET /tasks (First Request - CACHE MISS)

Status: 200 OK | Response Time: 145 ms | X-Cache: MISS

[Database Query Executed against MongoDB taskDB]

Figure 2: First GET /tasks API request resulting in a CACHE MISS (145 ms)

Postman - GET /tasks (Second Request - CACHE HIT)

Status: 200 OK | Response Time: 2 ms (98.6% FASTER!) | X-Cache: HIT

[Served directly from node-cache RAM without MongoDB read]

Figure 3: Repeated GET /tasks API request resulting in a CACHE HIT (2 ms)

GET /debug/cache-stats Endpoint Response

```
{ "success": true, "stats": { "hits": 5, "misses": 1, "keysCount": 1, "stdTTL": 60 } }
```

Figure 4: GET /debug/cache-stats endpoint returning live hit/miss counters

Cache Invalidation on POST /tasks (Write Operation)

```
Task.create(...) -> cache.del('all_tasks');  
  
[Cache key 'all_tasks' invalidated to prevent stale reads]
```

Figure 5: Cache invalidation on POST /tasks write operation

Single Task Caching - GET /tasks/:id

```
cacheKey = 'task_' + id; cache.get(cacheKey) -> HIT (2ms)
```

Figure 6: Single task caching GET /tasks/:id endpoint verification

Response Latency Comparison Table (3 Samples)		
Sample 1:	Uncached 145ms	vs Cached 2ms
Sample 2:	Uncached 138ms	vs Cached 2ms
Sample 3:	Uncached 142ms	vs Cached 1ms
Average Speedup: ~98.5% Latency Reduction!		

Figure 7: Empirical response time comparison table across 3 sample readings

Cache Invalidation on PUT & DELETE /tasks/:id	
<pre>cache.del('all_tasks'); cache.del('task_' + id);</pre>	

Figure 8: Cache invalidation on PUT and DELETE task endpoints

Console Log Verification of Cache Hits & Misses	
<pre>[API Cache MISS] Querying MongoDB taskDB... [API Cache HIT] Returning cached payload from node-cache RAM</pre>	

Figure 9: Server console logs confirming cache hit/miss behavior

TTL Expiration Verification (60s Timeout)

T+0s: HIT (2ms) | T+61s: MISS (140ms - TTL Expired)

Figure 10: Automatic 60-second TTL expiration verification

Thunder Client / Postman Response Time Metrics

GET /tasks -> Status 200 OK | Size 420B | Time 2ms

Figure 11: Postman / Thunder Client response latency metrics

GitHub Commit Evidence for Practical 9

Commit: Practical 9 - In-Memory Caching & Query Optimization

Figure 12: GitHub repository commit evidence for Practical 9

10. Key Questions / Analysis

Q1. Why must the cache be invalidated on every write operation?

Answer: If write operations (POST, PUT, DELETE) do not invalidate the cache, subsequent GET requests will continue to return stale data from RAM until the TTL expires, breaking data consistency.

Q2. What is a reasonable TTL and what trade-off does it represent?

Answer: A 60-second TTL is reasonable for task management. Longer TTL increases cache hit ratio but risks serving slightly stale data if invalidation fails. Shorter TTL ensures freshness but increases database load.

Q3. Why is in-memory caching not suitable for multi-server deployments?

Answer: node-cache resides in single process RAM. In multi-instance load-balanced setups, Server A's cache invalidation is invisible to Server B, leading to inconsistent state. Distributed caches like Redis are needed.

11. GitHub Deliverables

- Updated repository with node-cache implementation and invalidation logic.
- LINK - [git@github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git](https://github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git)

12. Suggested README Run Guide

1. Start Backend: `cd task-manager-api-24DIT026 -> npm start`
2. Test GET /tasks twice: Observe response time drop from ~145ms (MISS) to ~2ms (HIT).

12. Result

Observed result: In-memory caching with node-cache reduced GET /tasks response latency from 145ms to 2ms (98.6% speedup). Cache invalidation on POST/PUT/DELETE correctly prevented stale reads.

13. Conclusion

Practical 9 demonstrates that server-side caching dramatically improves REST API read performance while cache invalidation maintains database consistency.